

WIE

Follow Service & User Service

Complete API Documentation for React Native Developers

Follow Service: port 5009 • User Service: port 5005 • All routes require Bearer token • v1.0

Table of Contents

PART A — Follow Service (port 5009)

1. Service Overview & Base URL
2. Authentication
3. Follow Status & Data Model
4. Follow a User (POST /follow/:targetUserId)
5. Unfollow a User (DELETE /unfollow/:targetUserId)
6. Check Follow Status (GET /follow/status/:targetUserId)
7. Check Is-Following (GET /is-following/:targetUserId)
8. Check Is-Followed-By (GET /is-followed-by/:targetUserId)
9. Detailed Follow Status (GET /detailed-status/:targetUserId)
10. Follow Stats (GET /follow/stats/:userId)
11. Get Followers (GET /followers/:userId)
12. Get Following (GET /following/:userId)
13. Other Profile Followers (GET /other-followers/:userId)
14. Other Profile Following (GET /other-following/:userId)
15. Get Incoming Follow Requests (GET /follow-requests)
16. Accept Follow Request (POST /accept-request/:followerId)
17. Reject Follow Request (POST /reject-request/:followerId)
18. Cancel Sent Request (DELETE /cancel-request/:targetUserId)
19. Check Incoming Request Status (GET /follow-request/status/:fromUserId)
20. Get Sent Follow Requests (GET /get-send-follow-request/:userId)
21. Auto-Accept Pending Requests (POST /auto-accept-requests)
22. Full Follow Route Reference

PART B — User Service (port 5005) — Follow-Related & Profile

23. Account Privacy (GET & PUT /user/account-privacy)

- 24. User Search (GET /user/search)
- 25. Suggested Users (GET /user/suggested)
- 26. Get User by ID (GET /user/:userId)
- 27. Mute & Unmute Users
- 28. User Service — Full Route Reference
- 29. React Native Integration Examples

PART A — WIE Follow Service (Base URL: <http://localhost:5009/api>)

01 Service Overview & Base URL

The WIE Follow Service runs on port 5009 and manages all social connection logic between users — following, unfollowing, follow requests for private accounts, and all related stats. It is a standalone microservice that communicates with the User Service via gRPC for user details.

Service Config

```
Base URL (development) : http://localhost:5009/api
Base URL (Android emu) : http://10.0.2.2:5009/api

All routes share the prefix: /api
Example full URL: http://localhost:5009/api/follow/status/user_abc
```

► Public vs. Private Account Flow

Scenario	Behaviour
Public account	Following is instant. status='active'. No approval needed. Follower/following counts increment immediately.
Private account	A follow request is sent. status='pending'. Target must accept before status becomes 'active'. Counts only increment on accept.
Auto-accept	When a user switches from private to public, all pending requests are automatically accepted via POST /auto-accept-requests.

02 Authentication

Every endpoint in the Follow Service requires a Bearer token. The token is the same JWT obtained at login from the User Service. In React Native store the token in AsyncStorage and inject it via an Axios request interceptor.

TypeScript

```
// src/services/followService.ts - Axios instance
import AsyncStorage from '@react-native-async-storage/async-storage';
import axios from 'axios';
import Config from 'react-native-config';

const BASE = Config.FOLLOW_API_URL || 'http://10.0.2.2:5009/api';

export const followApi = axios.create({ baseURL: BASE, timeout: 15000 });

followApi.interceptors.request.use(async (cfg) => {
  const token = await AsyncStorage.getItem('token');
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  return cfg;
});
```

A 401 response means the token is missing or expired. Clear AsyncStorage and navigate to the Login screen.

03 Follow Status & Data Model

The Follow record has a status field that drives all follow-related UI logic. Understanding these values is essential for rendering the correct button state in your components.

status value	Meaning
active	User A is actively following User B. Counts are live.
pending	User A sent a follow request to private User B. Awaiting approval.
none	No follow relationship exists between the two users.

► Follow Record Object (returned in lists)

Field	Type	Required	Description
id	string	—	User ID of the follower or followed user.
name	string?	—	Display name of the user.
username	string?	—	Username handle (without @).
profile_picture	string?	—	URL of the user's profile picture. Null if not set.
bio	string?	—	User's bio text.
is_verified	boolean	—	True if user has a verified badge.
followedAt	string	—	ISO 8601 timestamp of when the follow occurred.
isFollowing	boolean	—	Whether the CURRENT viewer follows this user (on other-profile queries only).

04 Follow a User

Sends a follow request or instantly follows a user depending on their account privacy setting. The server determines public vs. private automatically — the client only sends the target user ID.

Method	Endpoint	Auth	Description
POST	/follow/:targetUserId	 Yes	Follow a user. Returns active (public) or pending (private) status.

► URL Parameter

Parameter	Description
:targetUserId	The user ID of the user to follow.

► Success Response — Public Account (HTTP 200)

JSON Response

```
{
  "success": true,
  "message": "User followed successfully",
  "status": "active",
  "requestStatus": "active",
  "isPrivateAccount": false,
  "data": {
    "followerId": "user_abc",
    "followingId": "user_xyz"
  }
}
```

► Success Response — Private Account (HTTP 200)

JSON Response

```
{
  "success": true,
  "message": "Follow request sent",
  "status": "pending",
  "requestStatus": "pending",
  "isPrivateAccount": true,
  "data": {
    "followerId": "user_abc",
    "followingId": "user_xyz"
  }
}
```

► Error Responses

Error Codes

```
400 ALREADY_FOLLOWING      - Already following this user
400 REQUEST_ALREADY_SENT   - Follow request already sent (pending)
400 CANNOT_FOLLOW_SELF     - Cannot follow yourself
500 INTERNAL_ERROR         - Unexpected server error
```

► React Native — Follow Button Logic

TypeScript

```
const handleFollowPress = async (targetId: string, currentStatus: string) => {
  if (currentStatus === 'active') {
    // Already following - show Unfollow confirmation
    return showUnfollowConfirmation(targetId);
  }
  if (currentStatus === 'pending') {
    // Request sent - tap cancels it
    await followApi.delete(`/cancel-request/${targetId}`);
    setFollowStatus('none');
    return;
  }
  // Not following - send request
  const res = await followApi.post(`/follow/${targetId}`);
  // res.data.status will be 'active' or 'pending'
  setFollowStatus(res.data.status);
};
```

Always call GET /follow/status/:targetUserId first when opening a profile screen to get the current follow state before rendering the button.

05 Unfollow a User

Removes an active follow relationship. Follower/following counts are decremented immediately. Attempting to unfollow someone you are not following returns 404.

Method	Endpoint	Auth	Description
DELETE	/unfollow/:targetUserId	 Yes	Unfollow a user. Decrements counts on both sides.

► Success Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "message": "User unfollowed successfully",
  "followerId": "user_abc",
  "followingId": "user_xyz"
}
```

► Error Responses

Error Codes

404	Not following this user	- No active follow record exists
400	Invalid operation	- Cannot unfollow yourself

06 Check Follow Status

Returns the current follow relationship between the authenticated user and any target user. This is the primary endpoint for determining which button to show on a profile screen.

Method	Endpoint	Auth	Description
GET	/follow/status/:targetUserId	✓ Yes	Get follow status between the caller and a target user.

► Success Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "isFollowing": true,      // true if status is 'active'
  "isPending": false,      // true if status is 'pending'
  "requestStatus": "active", // 'active' | 'pending' | 'none'
  "isSelf": false          // true if targetUserId === caller's own ID
}
```

► UI State Matrix

Response State	Button to Show
<code>isFollowing=true,</code> <code>isPending=false</code>	Show 'Following' button (tap to unfollow)
<code>isFollowing=false,</code> <code>isPending=true</code>	Show 'Requested' button (tap to cancel)
<code>isFollowing=false,</code> <code>isPending=false</code>	Show 'Follow' button (tap to follow)
<code>isSelf=true</code>	Hide follow button entirely — show 'Edit Profile'

07 Check Is-Following (Simple Boolean)

Lightweight endpoint that returns a single boolean. Use this when you only need to know if the caller follows the target — no extra state needed.

Method	Endpoint	Auth	Description
GET	/is-following/:targetUserId	 Yes	Returns true if caller follows the target user (status=active).

► Response

JSON Response

```
{ "success": true, "isFollowing": true }
```

08 Check Is-Followed-By

Checks whether the target user follows the caller. Useful for showing a 'Follows you' badge on a profile header.

Method	Endpoint	Auth	Description
GET	/is-followed-by/:targetUserId	 Yes	Returns true if the target user follows the caller.

► Response

JSON Response

```
{ "success": true, "isFollowedBy": true }
```

09 Detailed Follow Status

Returns extended follow relationship data in a single call — useful when you need all status fields at once without combining multiple endpoints.

Method	Endpoint	Auth	Description
GET	/detailed-status/:targetUserId	 Yes	Full follow relationship details between caller and target.

► Response

JSON Response

```
{
  "success": true,
  "isFollowing": true,
  "isPending": false,
  "requestStatus": "active",
  "status": "active"
}
```

10 Follow Stats

Returns the follower and following counts for any user. Used in the profile stats row. These counts reflect only active (accepted) follow relationships.

Method	Endpoint	Auth	Description
GET	/follow/stats/:userId	 Yes	Get follower and following counts for a user.

► Response

JSON Response

```
{
  "success": true,
  "userId": "user_xyz",
  "followers": 1240,
  "following": 380
}
```

11 Get Followers

Returns a paginated list of users who follow the specified user. Each entry includes enriched user data fetched from the User Service. Used on the self-profile to show your own followers.

Method	Endpoint	Auth	Description
GET	/followers/:userId	 Yes	Paginated list of followers for a user. Includes user details.

► Query Parameters

Field	Type	Required	Description
page	number	No	Page number (default: 1).
limit	number	No	Results per page (default: 20).

► Response

JSON Response

```
{
  "success": true,
  "total": 1240,
  "page": 1,
  "totalPages": 62,
  "followers": [
    {
      "id": "user_abc",
      "name": "Jane Doe",
      "username": "janedoe",
      "profile_picture": "https://cdn.../avatar.jpg",
      "bio": "Travel lover",
      "is_verified": false,
      "followedAt": "2025-11-01T12:00:00.000Z"
    }
  ]
}
```

12 Get Following

Returns a paginated list of users that the specified user follows. Same shape as the followers list. Used on the self-profile to show who you are following.

Method	Endpoint	Auth	Description
GET	/following/:userId	 Yes	Paginated list of users that userId follows.

► Query Parameters

Field	Type	Required	Description
page	number	No	Page number (default: 1).
limit	number	No	Results per page (default: 20).

► Response

JSON Response

```
{
  "success": true,
  "total": 380,
  "page": 1,
  "totalPages": 19,
  "following": [
    {
      "id": "user_xyz",
      "name": "John Smith",
      "username": "johnsmith",
      "profile_picture": "https://cdn.../avatar.jpg",
      "bio": "Dev @ WIE",
      "is_verified": true,
      "followedAt": "2025-10-15T08:00:00.000Z"
    }
  ]
}
```

13 Other Profile — Followers

Used when viewing ANOTHER user's profile. Returns their followers list AND includes isFollowing — whether the current authenticated viewer already follows each person in the list. This is critical for rendering Follow/Following buttons inside the list.

Method	Endpoint	Auth	Description
GET	/other-followers/:userId	 Yes	Another user's followers list with viewer's follow status per entry.

► Response — Difference from /followers/:userId

JSON Response

```
{
  "success": true,
  "total": 540,
  "followers": [
    {
      "id": "user_abc",
      "name": "Jane Doe",
      "username": "janedoe",
      "profile_picture": "https://cdn.../avatar.jpg",
      "is_verified": false,
      "followedAt": "2025-11-01T12:00:00.000Z",
      "isFollowing": true    // current viewer follows this person
    }
  ]
}
```

Use /other-followers and /other-following when viewing someone else's profile. Use /followers and /following on the self-profile. The difference is the isFollowing field per list item.

14 Other Profile — Following

Same as other-followers but returns who the target user follows. Each entry includes isFollowing to show if the current viewer also follows that person.

Method	Endpoint	Auth	Description
GET	/other-following/:userId	 Yes	Another user's following list with viewer's follow status per entry.

► Response

JSON Response

```
{
  "success": true,
  "total": 220,
  "following": [
    {
      "id": "user_xyz",
      "name": "John Smith",
      "username": "johnsmith",
      "isFollowing": false // viewer does NOT follow this person
    }
  ]
}
```

15 Get Incoming Follow Requests

Returns pending follow requests sent TO the authenticated user. Only relevant for private accounts. Each request includes the requester's profile details.

Method	Endpoint	Auth	Description
GET	/follow-requests	 Yes	Get all pending incoming follow requests for the current user.

► Query Parameters

Field	Type	Required	Description
page	number	No	Page number (default: 1).
limit	number	No	Results per page (default: 20).

► Response

JSON Response

```
{
  "success": true,
  "total": 5,
  "page": 1,
  "totalPages": 1,
  "requests": [
    {
      "id": "user_abc",
      "name": "Jane Doe",
      "username": "janedoe",
      "profile_picture": "https://cdn.../avatar.jpg",
      "bio": "Travel lover",
      "is_verified": false,
      "requestedAt": "2025-12-01T10:00:00.000Z"
    }
  ]
}
```

16 Accept Follow Request

Accepts a pending incoming follow request. The follow status changes from pending to active and follower/following counts are incremented on both sides.

Method	Endpoint	Auth	Description
POST	/accept-request/:followerId	 Yes	Accept a pending follow request from followerId.

► URL Parameter

Parameter	Description
:followerId	The user ID of the person who SENT the follow request to you.

► Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "message": "Follow request accepted",
  "followerId": "user_abc",
  "followingId": "user_xyz"
}
```

► Error Responses

Error Codes

```
404 No pending follow request found - Request does not exist or already accepted
```

17 Reject Follow Request

Permanently deletes a pending follow request. The requester is not notified. They can send a new follow request in the future.

Method	Endpoint	Auth	Description
POST	/reject-request/:followerId	 Yes	Reject (delete) a pending follow request.

► Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "message": "Follow request rejected",
  "followerId": "user_abc",
  "followingId": "user_xyz"
}
```

18 Cancel Sent Follow Request

Cancels a pending follow request that the authenticated user previously sent. Removes the Follow record entirely. The target user is not notified.

Method	Endpoint	Auth	Description
DELETE	/cancel-request/:targetUserId	 Yes	Cancel a pending outgoing follow request.

► Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "message": "Follow request cancelled",
  "followerId": "user_abc",
  "followingId": "user_xyz"
}
```

► Error Responses

Error Codes

```
404 No pending follow request found - Request does not exist or was already cancelled
```

19 Check Incoming Request Status

Checks whether a specific user has sent a follow request to the authenticated user, and whether the authenticated user is following them back. Useful for the Follow Requests screen.

Method	Endpoint	Auth	Description
GET	/follow-request/status/:fromUserId	 Yes	Check if fromUserId has a pending request to the caller.

► Response

JSON Response

```
{
  "success": true,
  "hasRequest": true,
  "requestStatus": "pending",    // 'pending' | 'active' | 'none'
  "isFollowingBack": false      // true if caller follows fromUserId
}
```

20 Get Sent Follow Requests

Returns all pending follow requests that the specified user has sent out but which have not yet been accepted. Used in the 'Requested' tab of the follow modal on self-profile.

Method	Endpoint	Auth	Description
GET	/get-send-follow-request/:userId	 Yes	Get all outgoing pending requests sent by userId.

► Response

JSON Response

```
{
  "success": true,
  "sentRequests": [
    {
      "id": "user_xyz",
      "userId": "user_xyz",
      "name": "John Smith",
      "username": "johnsmith",
      "profile_picture": "https://cdn.../avatar.jpg",
      "bio": "Dev @ WIE",
      "is_verified": false,
      "followedAt": "2025-12-01T10:00:00.000Z",
      "requestStatus": "pending"
    }
  ]
}
```

21 Auto-Accept Pending Requests

Bulk-accepts ALL pending incoming follow requests for the authenticated user in a single operation. Called automatically when a user switches their account from private to public. The server emits follow events for analytics but does NOT send individual notifications for each accepted request.

Method	Endpoint	Auth	Description
POST	/auto-accept-requests	 Yes	Bulk-accept all pending incoming requests. Called on private to public switch.

► Response (HTTP 200)

JSON Response

```
{
  "success": true,
  "message": "Auto-accepted 12 follow requests",
  "acceptedCount": 12
}
```

This endpoint is called automatically by the User Service when a user updates account privacy from 'private' to 'public'. You typically do NOT need to call this manually from the client.

22 Full Follow Route Reference

All routes are relative to `http://localhost:5009/api`. Every route requires a Bearer token. Routes are listed in the same order they appear in the router.

Method	Endpoint	Auth	Description
POST	/follow/:targetUserId	✓ Yes	Follow a user
DELETE	/unfollow/:targetUserId	✓ Yes	Unfollow a user
GET	/followers/:userId	✓ Yes	Self-profile follower list
GET	/following/:userId	✓ Yes	Self-profile following list
GET	/other-followers/:userId	✓ Yes	Other profile follower list + viewer's isFollowing
GET	/other-following/:userId	✓ Yes	Other profile following list + viewer's isFollowing
GET	/follow/status/:targetUserId	✓ Yes	isFollowing, isPending, requestStatus
GET	/follow/stats/:userId	✓ Yes	Follower and following counts
GET	/is-following/:targetUserId	✓ Yes	Simple boolean: does caller follow target
GET	/is-followed-by/:targetUserId	✓ Yes	Simple boolean: does target follow caller
GET	/follow-requests	✓ Yes	Incoming pending requests (private accounts)
POST	/accept-request/:followerId	✓ Yes	Accept a follow request
POST	/reject-request/:followerId	✓ Yes	Reject a follow request
DELETE	/cancel-request/:targetUserId	✓ Yes	Cancel an outgoing request
GET	/detailed-status/:targetUserId	✓ Yes	Extended follow status
GET	/follow-request/status/:fromUserId	✓ Yes	Check if fromUser sent me a request
GET	/get-send-follow-request/:userId	✓ Yes	Sent / outgoing pending requests
POST	/auto-accept-requests	✓ Yes	Bulk-accept all pending incoming requests

PART B — WIE User Service (Base URL: <http://localhost:5005/api/user>)

23 Account Privacy

Controls whether the user's account is public or private. This setting directly affects the Follow Service — private accounts receive follow requests instead of instant follows. Switching from private to public triggers auto-accept of all pending requests.

Method	Endpoint	Auth	Description
GET	/user/account-privacy	✓ Yes	Get the current user's account privacy setting.
PUT	/user/account-privacy	✓ Yes	Update privacy to 'public' or 'private'.

► GET /user/account-privacy — Response

JSON Response

```
{
  "success": true,
  "accountPrivacy": "public" // 'public' | 'private'
}
```

► PUT /user/account-privacy — Request Body

Field	Type	Required	Description
accountPrivacy	string	Yes	New privacy value. Must be exactly 'public' or 'private'.

► PUT /user/account-privacy — Response

JSON Response

```
{
  "success": true,
  "message": "Account privacy updated to private",
  "accountPrivacy": "private"
}
```

When switching private to public, the server automatically calls the Follow Service to bulk-accept all pending follow requests. No extra call needed from the client.

24 User Search

Searches users by name or username. Blocked and inactive users are excluded. The current user is excluded from results. Results are paginated.

Method	Endpoint	Auth	Description
GET	/user/search	 Yes	Search active users by name or username.

► Query Parameters

Field	Type	Required	Description
query	string	Yes	Search keyword (name or username). Minimum 1 character.
page	number	No	Page number (default: 1).
limit	number	No	Results per page (default: 20).

► Response

JSON Response

```
{
  "success": true,
  "total": 8,
  "page": 1,
  "totalPages": 1,
  "users": [
    {
      "id": "user_abc",
      "name": "Jane Doe",
      "username": "janedoe",
      "profile_picture": "https://cdn.../avatar.jpg",
      "bio": "Travel lover",
      "is_verified": false,
      "followers_count": 1240,
      "following_count": 380
    }
  ]
}
```

25 Suggested Users

Returns a list of suggested users to follow. Blocked and inactive users are excluded. The current user is excluded. Use this on the home screen or discover tab.

Method	Endpoint	Auth	Description
GET	/user/suggested	 Yes	Get suggested users to follow.

► Query Parameters

Field	Type	Required	Description
limit	number	No	Number of suggestions to return (default: 10).

► Response

JSON Response

```
{
  "success": true,
  "users": [
    {
      "id": "user_xyz",
      "name": "John Smith",
      "username": "johnsmith",
      "profile_picture": "https://cdn.../avatar.jpg",
      "bio": "Dev @ WIE",
      "is_verified": true,
      "followers_count": 5200
    }
  ]
}
```

26 Get User by ID

Fetches the full public profile of any user by their ID. Blocked and inactive users return 404. Used when navigating to another user's profile screen.

Method	Endpoint	Auth	Description
GET	/user/:userId	 Yes	Get full public profile of a user by ID.

► Response

JSON Response

```
{
  "success": true,
  "user": {
    "id": "user_xyz",
    "name": "John Smith",
    "username": "johnsmith",
    "email": "john@example.com",
    "profile_picture": "https://cdn.../avatar.jpg",
    "bio": "Dev @ WIE",
    "website": "https://johnsmith.dev",
    "gender": "male",
    "is_verified": true,
    "is_blocked": false,
    "status": "active",
    "accountPrivacy": "public",
    "isOnline": true,
    "last_seen_at": "2025-12-01T11:30:00.000Z",
    "followers_count": 5200,
    "following_count": 820,
    "posts_count": 148,
    "created_at": "2024-01-15T00:00:00.000Z"
  }
}
```

27 Mute & Unmute Users

Muting a user hides their content from your feed without unfollowing them. The muted user is not notified. Individual content types can be muted independently (posts, stories, reels, notes).

Method	Endpoint	Auth	Description
POST	/user/mute	✓ Yes	Mute a user with optional content-type filters.
POST	/user/unmute	✓ Yes	Unmute a previously muted user.
GET	/user/mutes	✓ Yes	Get the full list of users the caller has muted.
GET	/user/muted-count	✓ Yes	Get the total count of muted users.
GET	/user/mute-status/:userId	✓ Yes	Check if the caller has muted a specific user.

► POST /user/mute — Request Body

Field	Type	Required	Description
muted_user_id	string	Yes	User ID of the user to mute.
mute_posts	boolean	No	Mute their posts. Default: true.
mute_stories	boolean	No	Mute their stories. Default: true.
mute_reels	boolean	No	Mute their reels. Default: false.
mute_notes	boolean	No	Mute their notes. Default: false.

► POST /user/mute — Response

JSON Response

```
{
  "success": true,
  "message": "User muted successfully",
  "data": {
    "muterId": "user_abc",
    "mutedUserId": "user_xyz",
    "mutePosts": true,
    "muteStories": true,
    "muteReels": false,
    "muteNotes": false
  }
}
```

► POST /user/unmute — Request Body

Field	Type	Required	Description
muted_user_id	string	Yes	User ID of the user to unmute.

► GET /user/mute-status/:userId — Response

JSON Response

```
{
  "success": true,
  "data": {
    "isMuted": true,
    "mutePosts": true,
    "muteStories": true,
    "muteReels": false,
    "muteNotes": false
  }
}
```

28 User Service — Follow-Related Route Reference

All routes below are relative to `http://localhost:5005/api/user`. All require Bearer token unless marked Public.

Method	Endpoint	Auth	Description
GET	/get-profile	✓ Yes	Get the authenticated user's own full profile
PUT	/update-profile	✓ Yes	Update profile (multipart — name, bio, profile_picture)
GET	/account-privacy	✓ Yes	Get current privacy setting (public private)
PUT	/account-privacy	✓ Yes	Update privacy setting — triggers auto-accept if public
GET	/search?query=	✓ Yes	Search users by name or username
GET	/suggested	✓ Yes	Get suggested users to follow
GET	/:userId	✓ Yes	Get any user's public profile by ID
POST	/mute	✓ Yes	Mute a user (with content-type granularity)
POST	/unmute	✓ Yes	Unmute a previously muted user
GET	/mutes	✓ Yes	List all muted users
GET	/muted-count	✓ Yes	Count of muted users
GET	/mute-status/:user_id	✓ Yes	Mute status for a specific user
POST	/logout	✓ Yes	Invalidate server-side token
POST	/heartbeat	✓ Yes	Update online/last-seen status (call every 60 s)
PUT	/update-show-badge	✓ Yes	Toggle verified badge visibility
PUT	/update-show-suggestion	✓ Yes	Toggle whether user appears in suggestions

29 React Native Integration Examples

► Complete followService.ts

TypeScript

```
// src/services/followService.ts
import AsyncStorage from '@react-native-async-storage/async-storage';
import axios from 'axios';
import Config from 'react-native-config';

const followApi = axios.create({
  baseURL: Config.FOLLOW_API_URL || 'http://10.0.2.2:5009/api',
  timeout: 15000,
});

followApi.interceptors.request.use(async (cfg) => {
  const t = await AsyncStorage.getItem('token');
  if (t) cfg.headers.Authorization = `Bearer ${t}`;
  return cfg;
});

// Core actions
export const followUser = (id: string) =>
  followApi.post(`/follow/${id}`).then(r => r.data);

export const unfollowUser = (id: string) =>
  followApi.delete(`/unfollow/${id}`).then(r => r.data);

export const checkFollowStatus = (id: string) =>
  followApi.get(`/follow/status/${id}`).then(r => r.data);

export const getFollowStats = (id: string) =>
  followApi.get(`/follow/stats/${id}`).then(r => r.data);

// Lists
export const getFollowers = (id: string, page = 1, limit = 20) =>
  followApi.get(`/followers/${id}`, { params: { page, limit } }).then(r => r.data);

export const getFollowing = (id: string, page = 1, limit = 20) =>
  followApi.get(`/following/${id}`, { params: { page, limit } }).then(r => r.data);

export const getOtherFollowers = (id: string, page = 1) =>
  followApi.get(`/other-followers/${id}`, { params: { page } }).then(r => r.data);

export const getOtherFollowing = (id: string, page = 1) =>
  followApi.get(`/other-following/${id}`, { params: { page } }).then(r => r.data);

// Requests
export const getFollowRequests = (page = 1) =>
  followApi.get('/follow-requests', { params: { page } }).then(r => r.data);

export const getSentRequests = (userId: string) =>
  followApi.get(`/get-send-follow-request/${userId}`).then(r => r.data);

export const acceptRequest = (followerId: string) =>
```

```

    followApi.post(`/accept-request/${followerId}`).then(r => r.data);

export const rejectRequest      = (followerId: string) =>
    followApi.post(`/reject-request/${followerId}`).then(r => r.data);

export const cancelRequest      = (targetId: string) =>
    followApi.delete(`/cancel-request/${targetId}`).then(r => r.data);

```

► Profile Screen — Load Follow Data

TypeScript

```

// When opening any profile screen:
const loadProfileFollowData = async (targetUserId: string, isSelf: boolean) => {
    const [stats, status] = await Promise.all([
        getFollowStats(targetUserId),
        isSelf ? null : checkFollowStatus(targetUserId),
    ]);

    setFollowers(stats.followers);
    setFollowing(stats.following);

    if (!isSelf && status) {
        if (status.isSelf)           setButtonState('self');
        else if (status.isFollowing) setButtonState('following');
        else if (status.isPending)  setButtonState('requested');
        else                        setButtonState('follow');
    }
};

```

► Account Privacy Toggle — Settings Screen

TypeScript

```

const userApi = axios.create({
    baseURL: Config.API_BASE_URL || 'http://10.0.2.2:5005/api/user',
});
// inject token same way as followApi...

const togglePrivacy = async (newValue: 'public' | 'private') => {
    setLoading(true);
    try {
        const res = await userApi.put('/account-privacy', {
            accountPrivacy: newValue
        });
        // If switching to public, pending requests auto-accepted server-side
        setPrivacy(res.data.accountPrivacy);
        showToast(`Account is now ${res.data.accountPrivacy}`);
    } finally {
        setLoading(false);
    }
};

```

